

Komplett Lathund: Test kap. 1–4 (Teori, Syntax & Loopar)

1. God Praxis & Kodstil (PEP 8)

- **Namngivning:** Använd engelska, förklarande namn i `snake_case` (t.ex. `mean_velocity` istället för `v` eller `mean_vel`).
- **Hårdkodning:** Undvik värden/litteraler spridda i koden. Definiera dem en gång i tydligt namngivna variabler längst upp.
- **Globala variabler:** Undvik nyckelordet `global`. Skicka in data via funktionens parametrar och returnera värden med `return`.
- **Dokumentsträng (Docstring):** Skriv Beskrivning av funktionen" direkt under funktionshuvudet.

2. f-strings (Formaterade strängar): Vad betyder f och hur är syntaxen?

Bokstaven `f` framför en sträng står för **formatted string** (f-string).

Det gör att du kan bädda in Python-variabler och uttryck direkt inuti texten genom att omsluta dem med mäsvingar `{...}`. Python beräknar automatiskt värdet inuti `{...}` och omvandlar det till text vid utskrift.

```
namn = "Karolina"
alder = 22

# SÄTT 1: Med f-string (REN, TYDLIG OCH REKOMMENDERAD SYNTAX)
print(f"Hej! Jag heter {namn} och ar {alder} ar gammal.")

# SÄTT 2: Gamla / omständliga sättet (utan f-string)
print("Hej! Jag heter " + namn + " och ar " + str(alder) + " ar gammal.")

--- VAD DU KAN GÖRA INUTI { ... } ---

# 1. Direkt beräkning/matematik:
print(f"Om 5 ar ar jag {alder + 5} ar.") # Skriver ut: Om 5 ar ar jag 27 ar.

# 2. Anropa metoder eller funktioner:
print(f"Namn i stora bokstaver: {namn.upper()}") # Skriver ut: KAROLINA

# 3. Formatera flyttal/decimaler (t.ex. avrunda till 2 decimaler med {:.2f}):
pi = 3.14159265
print(f"Pi med tva decimaler: {pi:.2f}") # Skriver ut: 3.14

# 4. Formatera dictionaries:
bruker = {"namn": "Anna", "alder": 30}
# OBS: Anvand enkelfnuttar (') inuti mäsvingarna om strängen har dubbelfnuttar (") på utsidan!
print(f"Anvandaren heter {bruker['namn']}.")
```

3. Funktioner vs. Metoder med Punkt (.funktion())

- **Vanlig funktion** – `funktion(objekt)`: Du skickar in objektet som ett argument i funktionen.

```
len(min_lista) # Beräknar längden på listan
print("Hej")   # Skriver ut texten
type(x)        # Returnerar datatypen för x
```

- **Metod med punkt** – `objekt.metod()`: Metoden "hör" till objektets datatyp. Du ber objektet utföra en handling på sig själv.

```
text = "katt"
text.upper() # Returnerar "KATT" (strängmetod)
lista.append("hund") # Lägger till i slutet av listan (listmetod)
lager.get("applen") # Hämtar värde säkert från dictionary
```

4. Syntax för Loopar (for, while & range)

- **Hur range() fungerar:** range(start, stop, step) genererar en sekvens av tal. **Viktigt:** Sluttalet (stop) ingår **ALDRIG!**

- range(5) → 0, 1, 2, 3, 4
- range(2, 6) → 2, 3, 4, 5
- range(0, 10, 2) → 0, 2, 4, 6, 8 (steglängd 2)

- **for-loop:** Används när du vet hur många iterationer som ska köras, eller när du vill gå igenom varje element i en sekvens.

```
# 1. Gå igenom element i en lista
djur_lista = ["katt", "hund", "kanin"]
for djur in djur_lista:
    print("Djur:", djur)

# 2. Loopa med range(start, stop, step)
for i in range(0, 10, 2): # Går från 0 till 8 med steg 2
    print("Tal:", i)
```

- **while-loop med True, input() och break:** Används för kontinuerlig inmatning tills användaren aktivt avbryter (t.ex. genom att skriva "exit").

```
while True:
    svar = input("Ange ett ord (eller 'exit' för att avsluta): ")
    if svar.lower() == "exit":
        print("Avslutar programmet...")
        break # Hoppa direkt ut ur loopen
    print("Du skrev:", svar)
```

- Extra om for vs while loop. Användning av Index: for-loop omgjord till while-loop Detta exempel visar hur man går igenom en lista med index via en for-loop respektive hur man skriver om samma logik med en while-loop där indexet täcks med i = 0 och i += 1.

```
frukter = ["äpple", "banan", "päron"]

# SÄTT 1: for-loop med range(len(...))
for i in range(len(frukter)):
    print(f"Index {i}: {frukter[i]}")

# SÄTT 2: Omgjord till while-loop
i = 0 # 1. Starta räknaren på index 0
while i < len(frukter): # 2. Loopa så länge i är mindre än listans längd
    print(f"Index {i}: {frukter[i]}")
    i += 1 # 3. GLÖM INTE att öka räknaren varje varv!
```

- **break, continue och pass:**

- **break:** Avbryter loopen helt och hoppar ut.
- **continue:** Hoppa över resten av nuvarande varv och går till nästa iteration.
- **pass:** Platshållare för framtida kod (gör ingenting).

```
for tal in range(5):
    if tal == 2:
        continue # Hoppa över utskriften för talet 2
    if tal == 4:
        break # Avbryter loopen helt när talet är 4
    print(tal)
```

5. Nyckelordet in

Nyckelordet `in` används i två huvudsakliga sammanhang:

1. **Medlemskapstest (booleskt resultat):** Kontrollerar om ett element finns i en lista, sträng eller dictionary (returnerar `True` eller `False`).
2. **Iteration:** Används i `for`-loopar för att gå igenom sekvenser.

```
frukter = ["äpple", "banan", "päron"]

# 1. Medlemskapstest (blir True/False)
finns_banan = "banan" in frukter # True
har_bokstav = "a" in "katt"      # True
finns_nyckel = "applen" in {"applen": 5} # True

# 2. Iteration i for-loop
for frukt in frukter:
    print(frukt)
```

6. Operatörer, Modulo, Booleska Uttryck & Fallgropar

- **Modulo-operatören (%):** Beräknar **resten** vid heltalsdivision. Används ofta för att kontrollera om ett tal är jämnt eller udda.

```
tal = 7
if tal % 2 == 0:
    print("Talet är jämnt")
else:
    print("Talet är udda")
```

- **Uttryck som blir True eller False direkt:** Undvik onödiga `if/else`-satser för att returnera booleska värden.

```
# DÅLIG STIL:
def is_longer_bad(word):
    if len(word) > 8:
        return True
    else:
        return False

# BRA STIL (returnerar uttryckets booleska resultat direkt):
def is_longer(word):
    return len(word) > 8
```

- **Reserverade ord (Keywords):** Får ej användas som variabler: `and`, `if`, `for`, `True`, `False`, `None`, `break`, `continue`, `pass`, `def`, `return`, `with`.
- **Identifierare:** Måste börja med en bokstav eller understreck (`_`), aldrig med en siffra.
- **Vanliga operatorfel:** Potens skrivs med `**` (inte `^`), större än/lika med skrivs `>=` (inte `=>`), och jämförelse är `==` (inte `=`).
- **Flyttalsprecision:** Jämför inte flyttal med `==` (p.g.a. `0.1 + 0.2 != 0.3`). Använd en tolerans: `abs((0.1 + 0.2) - 0.3) < 1e-9`.

7. Teoribegrepp & Filhantering

- **Funktionsstruktur:**
 - *Huvud:* `def min_funktion(param1, param2=10):`
 - *Kropp:* Den indenterade koden i funktionen.
 - *Parametrar vs Argument:* Parametrar är variablerna i funktionens definition; Argument är de faktiska värdena som skickas vid anrop.
- **Paradigmer:** *Imperativ* (steg-för-steg ändring av tillstånd), *Funktionell* (rena funktioner utan sideeffekter), *OOP* (klasser och objekt med attribut/metoder).

- **Filhantering** – with open(...) vs. enbart open(...):

- Enbart open(): Kräver manuell .close(). Om programmet kraschar stängs inte filen.
- with open(...) (**REKOMMENDERAT**): Stänger filen automatiskt när blocket avslutas, vilket frigör resurser och säkrar sparandet.

```
# UTAN with (Dålig praxis - kräver fil.close()):
fil = open("brev.txt", "r")
innehall = fil.read()
fil.close()

# MED with (Bra praxis - stänger automatiskt):
with open("brev.txt", "r") as fil:
    innehall = fil.read()
```

- **System-strömmar:** sys.stdin (inmatning/tangentbord), sys.stdout (utmatning/konsol), sys.stderr (felmeddelanden).
- **Lokal kopia:** Använd .copy() för listor/objekt för att skapa en ny oberoende kopia i minnet så att ändringar inte påverkar originalet.

8. Snabbreferens för Datastrukturer

- **Listor (Muterbara/Ändringsbara):**

```
data = ["katt", 7, "joker"]
data.append("hund")      # Lägg till sist
data.insert(1, "apa")    # Lägg till på index 1
data.pop(2)              # Ta bort och returnera element på index 2
del data[0]              # Ta bort element på index 0
data.remove("joker")     # Ta bort första förekomsten av "joker"
data.reverse()           # Ändra listan på plats
baklänges = data[::-1]  # Ändra listan med slicing
```

- **Dictionaries (Uppslagstabeller):**

```
lager = {"applen": 10, "bananer": 5}
lager["bananer"] = 8      # Uppdatera värde
antal = lager.get("applen") # Hämtar värde säkert
for frukt, antal in lager.items(): # Loopa över nyckel och värde
    print(frukt, antal)
```

Extra Quiz Questions and answers

Your questions

1. When using int, float and str, can they be used together in calculations or other operations? If so, when do I need to convert between them, and which conversions are allowed?

Answer: In Python, you can use int, float, and str together in certain operations, but you need to be careful about the types.

- You can perform arithmetic operations between int and float without any issues. Python will automatically convert the int to a float if necessary. For example, '3 + 2.5' will yield '5.5'.

- However, you cannot directly perform arithmetic operations between str and int/float. If you want to combine a string with a number, you need to convert the number to a string using 'str()'. For example, 'str(3) + apples' will yield '"3 apples"'.

- Conversely, if you have a string that represents a number (like '"3"'), you can convert it to an int or float using 'int()' or 'float()'. For example, 'int("3") + 2' will yield '5'.

- The main issue is when it comes to how the computer stores data. It stores it in binary and can max take around 15-17 digits of precision for floats. So if you do a float + float == float then it will be true, but if you do a float + float == int then it will be false because the float is not exactly equal to the int. So you need to be careful when comparing floats and ints.

2. Could we focus on problem 10 today and practise nested for loops? I am unsure how to structure one loop inside another, what each loop controls and how to calculate the sum while going through every element in the matrix.

Answer: Nested for loops are used when you need to iterate over multi-dimensional data structures, like matrices (2D lists). The outer loop typically iterates over the rows, while the inner loop iterates over the columns of each row. To calculate the sum while going through every element, you can initialize a variable before the loops and then add each element to it inside the inner loop. For example:

```
matrix = [[1, 2, 3], [4, 5, 6]]
total = 0
for row in matrix:
    for value in row: # Even if it's every element in each column of the row it is
                      # written as "value in row" because the inner loop is iterating over each element
                      # in the current row.
        total += value
print(total) # Output: 21
```

You can also build a matrix with this method:

```
matrix = []
num = 1
for i in range(3): # Outer loop for rows
    row = []
    for j in range(4): # Inner loop for columns
        row.append(num) # This adds current nr: 1, 2, 3, 4
        num += 1
    matrix.append(row) # This adds row to matrix: 1, 2, 3
```

In math this would be $\sum_{i=0}^2 \sum_{j=0}^3 num_{ij} = 30$ to get the sum of the matrix.

3. For problem 9, I do not understand how the information from the original file is stored or how to change it before writing it into a new file. Could we go through the difference between `read()`, `readreadline()` and `readlinesand` how `replace()` is used on the contents of a file?

Answer: When you read from a file in Python, you have several methods to choose from:

- `'read()'`: This method reads the entire content of the file as a single STRING.
- `'readline()'`: This method reads the next LINE from the file.
- `'readlines()'`: This method reads all the LINES of the file into a LIST of string.
- `'replace()'`: This method can be used on the string returned by `'read()'` or `'readline()'` to replace parts of the content. For example:

```
with open("brev.txt", "r") as fil:
    innehall = fil.read()
innehall = innehall.replace("gammalt", "nytt")
with open("brev_nytt.txt", "w") as fil:
    fil.write(innehall)
```

4. For problem 6, I do not understand how start, stop and step are passed into the function or how the generated x-values are then used to calculate the y-values. Could we go through how the function's output becomes the data used by matplotlib and how to make sure the x- and y-value lists match correctly?

Answer: When you define a function that takes `'start'`, `'stop'`, and `'step'` as parameters, you can use these values to generate a list of x-values using the `'range()'` function or a list comprehension (which is when you create a list in a single line of code, e.g., `[x for x in range(start, stop, step)]`). The generated x-values can then be used to calculate corresponding y-values based on a mathematical function.

For example, if you want to calculate y-values for a linear function `'y = 2x + 1'`, you can do the following:

```
def generate_linear_data(start, stop, step):
    x_values = list(range(start, stop, step))
    y_values = [2 * x + 1 for x in x_values]
    return x_values, y_values
```

```
x, y = generate_linear_data(0, 10, 1)
print(x) # Output: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print(y) # Output: [1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
```

When you pass the 'x' and 'y' lists to matplotlib for plotting, it will plot the points correctly as long as both lists have the same length. The 'x-values' and 'y-values' are generated in a way that ensures they correspond to each other, so each x-value has a matching y-value.

Övningsuppgifter: Kap 1–4

Del 1: Hitta fel och förbättra koden (4 uppgifter)

Övning 1 (Kodstil, Globala variabler & Namngivning) Identifiera minst tre fel/brister mot PEP 8 och god praxis i koden nedan och skriv om koden så att den följer god praxis.

```
x = 1.25 # MOMS
def calc(v):
    global x
    res = v * x
    return res
print("Total:", calc(100))
```

Övning 2 (Boolska uttryck & Flyttalsprecision) Identifiera två fel/brister i koden (dålig stil samt en logisk fallgrop vid jämförelse) och skriv en korrelerad version.

```
def is_equal_and_long(val1, val2, text):
    if (val1 + val2) == 0.3:
        if len(text) > 5:
            return True
        else:
            return False
    return False
```

Övning 3 (Filhantering & Metoder/Strängar) Koden nedan har problem med resurshantering och föråldrad strängformatering. Hitta felen och skriv om med `with open` och f-string.

```
f = open("data.txt", "r")
text = f.read
print("Filens innehåll är: " + text + " och är klar.")
f.close()
```

Övning 4 (Loopar, Index & Oändlig loop) Koden ska skriva ut alla element i listan med deras index via en `while`-loop, men innehåller ett allvarligt syntax/logikfel. Hitta felet och rätta till det.

```
items = ["a", "b", "c"]
i = 0
while i < len(items):
    print(f"Index {i}: {items[i]}")
    # Saknar något här!
```

Del 2: Tillämpning och kodskrivning (6 uppgifter)

Övning 5 (f-strings & Dictionaries) Skriv ett skript (max 10 rader) som har en dictionary `user = {"name": "änna", "score": 8.1234}`. Skriv ut texten "User ANNA scored 8.12 points." med en f-string där namnet blir med stora bokstäver och poängen avrundas till 2 decimaler.

Övning 6 (Funktioner vs. Metoder) Skriv ett skript (max 10 rader) som skapar en lista med tre djur. Använd en inbyggd funktion för att ta reda på listans längd, och en listmetod för att lägga till ett nytt djur i slutet.

Övning 7 (for-loop med range) Skriv en for-loop med `range(start, stop, step)` som räknar nedåt från 10 till 2 (inklusive 2) med steglängd 2 och skriver ut varje tal.

Övning 8 (while-loop med break och continue) Skriv en `while` `True`-loop som ber om ett tal från användaren. Om talet är ett jämnt tal, hoppa över utskriften. Om talet är större än 100, avbryt loopen.

Övning 9 (Omvandla for till while) Omvandla följande `for`-loop till en `while`-loop med manuell indexräknare (`i = 0`):

```
colors = ["red", "green", "blue"]
for i in range(len(colors)):
    print(colors[i])
```

Övning 10 (Nyckelordet in & Säker hämtning) Skriv ett skript som skapar en dictionary `stock = {"apples": 5}`. Kontrollera om `bananas` finns i nycklarna, och hämta antalet bananer utan att koden kraschar om nyckeln saknas.

Lösningsförslag: Kap 1–4

Del 1: Hitta fel och förbättra koden

Lösning Övning 1 Fel: Otydliga variabelnamn (`x`, `v`, `res`), användning av `global`, avsaknad av docstring och hårdkodad global variabel i gemener.

Förbättrad kod:

```
VAT_RATE = 1.25 # Global konstant längst upp

def calculate_total_price(net_price):
    """Beräknar totalpriset inklusive moms."""
    return net_price * VAT_RATE

total_price = calculate_total_price(100)
print(f"Total: {total_price}")
```

Lösning Övning 2 Fel: Onödig `if/else` för booleskt returvärde samt direkt jämförelse av flyttal med `==` (`0.1 + 0.2 != 0.3` i Python).

Förbättrad kod:

```
def is_equal_and_long(val1, val2, text):
    """Jämför flyttal säkert med tolerans och returnerar uttryck direkt."""
    is_sum_correct = abs((val1 + val2) - 0.3) < 1e-9
    return is_sum_correct and len(text) > 5
```

Lösning Övning 3 Fel: Saknade parenteser vid metodanrop (`f.read` istället för `f.read()`), omständlig strängkonkatering samt avsaknad av `with open` (vilket gör att filen inte stängs om ett fel uppstår).

Förbättrad kod:

```
with open("data.txt", "r") as file_handle:
    content = file_handle.read()
    print(f"Filens innehåll är: {content} och är klar.")
```

Lösning Övning 4 Fel: Räknaren i inkrementeras inte, vilket leder till en oändlig loop.

Förbättrad kod:

```
items = ["a", "b", "c"]
i = 0
while i < len(items):
    print(f"Index {i}: {items[i]}")
    i += 1 # Viktigt: öka räknaren varje varv
```

Del 2: Tillämpning och kodskrivning

```
user = {"name": "anna", "score": 8.1234}
formatted_text = f"User {user['name'].upper()} scored {user['score']:.2f} points."
print(formatted_text)
```

```
animals = ["katt", "hund", "kanin"]
list_length = len(animals) # Inbyggd funktion len()
animals.append("marsvin") # Listmetod .append()
print(f"Längd: {list_length}, Lista: {animals}")
```

```
for number in range(10, 1, -2):
    print(f"\"Tal: {number}\"")
```

```
while True:
    number = int(input("Ange ett tal: "))
    if number % 2 == 0:
        continue # Hoppa över jämna tal
    if number > 100:
        break # Avbryt om talet är över 100
    print(f"Udda tal: {number}")
```

```
colors = ["red", "green", "blue"]
i = 0
while i < len(colors):
    print(colors[i])
    i += 1
```

```
stock = {"apples": 5}
has_bananas = "bananas" in stock # Blir False
banana_count = stock.get("bananas", 0) # Har en nolla i input för att undvika KeyError
# genom att returnera 0 om nyckeln saknas
print(f"Finns bananer? {has_bananas}. Antal: {banana_count}")
```